Day 13 - Practical: Project Structure, Git Workflow, Deployment

Day 13 — Practical: Project Structure, Git Workflow, Deploying a Static Site

Objectives

- Organize a multi-page site's files and folders the conventional way
- Practice a real git workflow, applied to a site project rather than just abstractly
- Understand what a "static site" is and deploy one for real, for free, so it's viewable by anyone on the internet

Organizing a real, multi-page site's files

For everything you've built so far in this course, one HTML file and one CSS file (or a small handful) was enough. A real, multi-page site needs a predictable structure so that images, styles, and pages can all find each other correctly:

```
my-site/
■■■ index.html            -- the homepage; browsers automatically look for this filename by default
■■■ about.html
■■■ contact.html
■■■ css/
■   ■■■ styles.css
■■■ images/
■   ■■■ logo.png
■   ■■■ hero-photo.jpg
■■■ README.md
```

`index.html` is a special, conventional filename: when a visitor navigates to a folder without specifying an exact file (e.g., just `https://example.com/`, or, locally, just opening the `my-site` folder), the browser (or web server) automatically looks for and serves `index.html` from that folder by default. This is why the homepage is always named `index.html`, and why you've likely already seen this filename recommended without a full explanation before now.

Referencing files across this structure uses **relative paths** — a path written relative to the *current* file's own location, rather than a full absolute address:

```
<!-- inside about.html, at the top level -->
<link rel="stylesheet" href="css/styles.css">
<img src="images/logo.png" alt="Company logo">
<a href="index.html">Home</a>
<a href="contact.html">Contact</a>
```

`css/styles.css` means "look inside a folder called `css`, sitting right next to this HTML file, for a file called `styles.css`." Get comfortable with relative paths now — a genuinely common beginner bug is an image or stylesheet that works when you open a file directly, but breaks the moment you upload the exact same project

to a real server, purely because a path assumed a folder structure that wasn't actually preserved consistently.

A real git workflow, applied to an HTML/CSS project

The core git commands and reasoning here directly mirror the same discipline used in any programming project — the practice, not the tool, is what genuinely matters:

```
git init                                    # once, at the start of the project
git add .
git commit -m "Initial commit: homepage structure and base styles"

# ... later, after adding a new page ...
git status                                    # see what's changed
git diff                                       # see the EXACT lines changed
git add about.html css/styles.css
git commit -m "Add About page and its styles"

git log --oneline                                 # see the project's history
```

A commit message should explain *why* a change was made, not simply restate *what* changed — `git diff` already shows precisely *what* changed in detail; a message like `"Fix navbar overlapping on mobile"` is far more useful to you later than a vague `"update css"`. Just as in any other kind of project, work in small, logical commits as you build each page or feature, rather than one enormous commit at the very end covering the entire two weeks of work — this gives you (and, on a real team, any collaborator) a genuinely useful history to look back through later.

Branches, exactly as in any other project: create a new branch for a meaningful chunk of work (`git checkout -b feature/add-contact-page`), commit your changes there, and merge back into your main branch once it's done and working — rather than committing every single change directly onto `main`. This habit matters just as much for a website project as for any other kind of software project, and it's worth practicing here rather than treating it as something that only applies to "real programming."

Deploying a static site — making it viewable by anyone on the internet

A **static site** is one made entirely of files that don't change based on who's requesting them or need a server to run any code — exactly what you've built throughout this entire course: plain HTML, CSS, and (if you add it later) client-side JavaScript, with no database or server-side logic involved. Static sites are the simplest kind of website to deploy, and several free services exist specifically for hosting them. **GitHub Pages** is a common, beginner-friendly choice, since it's directly built into GitHub (which you're likely already using for the git workflow above):

1. Push your project to a GitHub repository (create one on github.com, then `git remote add origin <your-repo-url>` and `git push -u origin main`, if you haven't already connected your local repository to one).
2. On GitHub, go to your repository's **Settings** tab, then find the **Pages** section.
3. Under "Source," choose the branch (usually `main`) and folder (usually `/ (root)`, unless your `index.html` lives in a subfolder) you want published.

4. GitHub builds and publishes your site at a URL in the form `https://your-username.github.io/your-repo-name/` — usually within a minute or two.

Once deployed this way, any future change follows a simple, repeatable cycle: make your edit locally, test it by opening the file directly in your browser (exactly as you've done all course), commit and push it to GitHub, and the live site updates automatically shortly afterward — no separate, manual "upload" step required beyond your normal `git push`.

Exercises

This is a hands-on, do-it-for-real day rather than an auto-graded one, using the Day 7 portfolio project (or a copy of it) as your subject:

1. Reorganize your Day 7 portfolio into the folder structure shown above (`index.html` at the root, a `css/` folder, an `images/` folder if you used any real images), updating every path reference to match.
2. Confirm the reorganized site still opens and works correctly by opening `index.html` directly in your browser.
3. If you have git installed: run `git init` in the project folder, then make at least two separate, meaningfully-described commits as you reorganize (one for the restructuring itself, and at least one more for some small additional change or fix you make along the way).
4. If you have a GitHub account: push the project to a new repository and enable GitHub Pages for it, following the steps above, then visit the live URL GitHub gives you and confirm your site actually works when loaded from the real internet, not just locally.

There's no separate `solution/` for this day — the "solution" is your own reorganized Day 7 project, genuinely deployed.